First Move Robotics

Board Interpreter Documentation

Version 0.2.0 (Development Build)

Strydr Silverberg
strydr_silverberg@mines.edu

Colorado School of Mines, Class of 2026

February 27, 2026

Board Interpreter: Physical move planning and path generation for the Kirin autonomous chess system

2026 Colorado School of Mines ProtoFund Recipient

Team: Strydr Silverberg, Colin Dake

Contents

1 Introduction **4**
1.1 Overview . . . 4
1.2 Responsibilities . . . 4
1.3 Position in the System Architecture . . . 4
1.4 Coordinate System . . . 5

2 Data Structures **6**
2.1 BoardCoord . . . 6
2.1.1 Member Functions . . . 6
2.1.2 Ordering . . . 7
2.2 Path . . . 7
2.3 PieceType . . . 7
2.4 PhysicalMove . . . 7
2.5 RelocationPlan . . . 8
2.6 MovePlan . . . 8
2.7 PhysicalBoard . . . 8

3 Path Generation **10**
3.1 Overview . . . 10
3.2 Dispatch . . . 10
3.3 Piece-Specific Generators . . . 10
3.3.1 Pawn . . . 10
3.3.2 Knight . . . 10
3.3.3 Bishop . . . 11
3.3.4 Rook . . . 11
3.3.5 Queen . . . 11
3.3.6 King . . . 11
3.4 Primitive Path Helpers . . . 11
3.5 Path Selection . . . 12

4 Blocker Detection and Relocation **13**
4.1 Blocker Detection . . . 13
4.2 Parking Spot Selection . . . 13
4.3 Recursive Blocker Relocation . . . 13

5 A* Pathfinding **15**
5.1 Overview . . . 15
5.2 Algorithm Details . . . 15
5.3 Distance Metrics . . . 15

6 Move Planning **17**
6.1 planMove . . . 17

6.2 Planning Algorithm . . . 17
6.3 Error Handling . . . 17

7 Utility Functions and Bitboard Helpers **19**
7.1 Neighbour Enumeration . . . 19
7.2 Inline Helper . . . 19
7.3 Bitboard Utilities . . . 19
7.4 Position Parsing . . . 19

8 Debug Functions **21**
8.1 printPath . . . 21
8.2 printMovePlan . . . 21

9 Testing **22**
9.1 Test Coverage . . . 22
9.2 Running Tests . . . 22

10 Integration Guide **23**
10.1 Typical Usage Pattern . . . 23
10.2 Capture Moves . . . 23
10.3 Special Chess Moves . . . 24
10.3.1 Castling . . . 24
10.3.2 En Passant . . . 24
10.3.3 Promotion . . . 24

A Glossary **25**

B References **26**

Chapter 1

Introduction

1.1 Overview

This document provides comprehensive documentation for the **Board Interpreter** module of the Kirin autonomous chess system. The board interpreter is the translation layer between the abstract chess world of the Kirin engine and the physical reality of the robotic gantry that moves pieces on the board.

When the Kirin engine decides on a move, it communicates that decision as a standard algebraic chess move (e.g., `e2e4`). The board interpreter's job is to answer the question: *how does the robot actually execute that move on a physical board with pieces that may be in the way?* This involves computing geometrically valid physical paths for the magnetic gantry head, detecting pieces that block those paths, temporarily relocating blockers to safe parking squares, and finally generating a complete ordered action plan for the gantry controller to execute step by step.

1.2 Responsibilities

The board interpreter module is responsible for:

- **Coordinate translation**: Converting between FEN algebraic notation and physical `(row, col)` grid coordinates.
- **Physical board state tracking**: Maintaining a temporary occupancy model of the board during move planning so that blocker relocations can be simulated before being sent to the gantry.
- **Piece-type-aware path generation**: Computing one or more geometrically valid sweep paths for each piece type (pawns, knights, bishops, rooks, queens, and kings), reflecting how the gantry head must physically travel.
- **Blocker detection and relocation planning**: Identifying pieces that obstruct the chosen path and computing sub-plans to park them temporarily on safe empty squares.
- **A* pathfinding for blocker routing**: Finding collision-free routes for blocker pieces being moved to and from parking spots, using A* search with Chebyshev distance as the heuristic.
- **Move plan assembly**: Packaging the complete sequence of relocations, the primary piece move, and post-move restorations of parked pieces into a `MovePlan` structure that the gantry controller can execute directly.

1.3 Position in the System Architecture

The board interpreter sits between the chess engine and the gantry controller in the Kirin software stack:

The engine provides two pieces of information: the move in UCI notation and a bitboard encoding which squares are currently occupied. The board interpreter uses these to produce a fully sequenced `MovePlan`. The gantry controller then executes each step of that plan in order without needing to reason about board geometry.

1.4 Coordinate System

The board interpreter uses a `(row, col)` coordinate system that maps directly to the physical grid layout of the board:

Axis	Range	Convention
Row	0–7	Row 0 = rank 8 (far side from white), Row 7 = rank 1
Col	0–7	Col 0 = file a (left side), Col 7 = file h (right side)

Table 1.1: Physical board coordinate conventions

Square indices used in the underlying bitboard follow the engine convention: bit 0 corresponds to `a8` (row 0, col 0) and bit 63 corresponds to `h1` (row 7, col 7). The formula is `square = row * 8 + col`.

Chapter 2

Data Structures

2.1 BoardCoord

`BoardCoord` is the fundamental coordinate type used throughout the module. It represents a single square on the physical board.

```
struct BoardCoord {
    int row;
    int col;

    BoardCoord();
    BoardCoord(int r, int c);

    bool operator==(const BoardCoord& other) const;
    bool operator!=(const BoardCoord& other) const;
    bool operator<(const BoardCoord& other) const;

    int toSquareIndex() const;
    static BoardCoord fromSquareIndex(int square);
    static BoardCoord fromFEN(const char* str);
    void toFEN(char* out) const;
};
```

Listing 2.1: BoardCoord definition

2.1.1 Member Functions

`toSquareIndex() const` *Method*
Parameters: None
Returns: int – square index in range [0, 63]

Converts the `(row, col)` coordinate to a linear square index using the formula `row * 8 + col`. This index is compatible with the bitboard representation used by the Kirin engine.

`fromSquareIndex(int square)` *Static Method*
Parameters: square – linear index [0, 63]
Returns: BoardCoord

Constructs a `BoardCoord` from a square index. Inverse of `toSquareIndex`.

`fromFEN(const char* str)` *Static Method*
Parameters: str – two-character FEN string, e.g. `"e4"`
Returns: BoardCoord

Parses a FEN algebraic square name into physical `(row, col)` coordinates. The file character (`a`–`h`) maps to column 0–7 and the rank digit (`1`–`8`) is flipped to row 7–0 respectively. Returns `BoardCoord(0,0)` on invalid input.

`toFEN(char* out) const` *Method*
Parameters: out – output buffer of at least 3 bytes
Returns: void

Serialises the coordinate back to a two-character FEN algebraic string followed by a null terminator. Inverse of `fromFEN`.

2.1.2 Ordering

The `operator<` overload allows `BoardCoord` values to be stored in `std::set` and `std::map` containers. Ordering is row-major: coordinates are compared first by row, then by column.

2.2 Path

A `Path` is an ordered sequence of squares that the gantry head must visit while carrying a piece. The first element is the first square entered *after* the starting position, and the last element is the destination square.

```
struct Path {
    std::vector<BoardCoord> squares;

    void append(const BoardCoord& coord);
    void clear();
    int length() const;
    bool empty() const;
    BoardCoord destination() const;
};
```

Listing 2.2: Path definition

> **Note:** Paths do not include the source square. They contain every intermediate square the gantry crosses plus the final destination, making the length equal to the number of squares traversed (not the chess-move distance).

2.3 PieceType

An enum identifying the type of piece involved in a move. The board interpreter uses this to select the appropriate path generation algorithm.

```
enum PieceType {
    PAWN, KNIGHT, BISHOP, ROOK, QUEEN, KING, UNKNOWN
};
```

Listing 2.3: PieceType enum

When `PieceType` is `UNKNOWN`, the path generator falls back to inferring the movement style from the geometric relationship between source and destination squares.

2.4 PhysicalMove

Encapsulates a single physical piece movement: where a piece starts, where it ends, what type it is, and whether the move is a capture.

```
struct PhysicalMove {
    BoardCoord from;
    BoardCoord to;
```

```
    PieceType  pieceType;
    bool       isCapture;
};
```

Listing 2.4: PhysicalMove definition

2.5 RelocationPlan

Represents the plan to temporarily move a single blocking piece out of the way of a primary piece move. It records both the source and destination parking square, and the A*-computed path the gantry will follow.

```
struct RelocationPlan {
    BoardCoord from;
    BoardCoord to;
    Path       path;
};
```

Listing 2.5: RelocationPlan definition

2.6 MovePlan

The top-level output of the board interpreter. A `MovePlan` is a fully ordered execution plan for one chess move on the physical board.

```
struct MovePlan {
    std::vector<RelocationPlan> relocations;   // blocker moves (pre-
        primary)
    PhysicalMove                primaryMove;    // the chess move
        itself
    Path                        primaryPath;    // gantry path for
        primary piece
    std::vector<RelocationPlan> restorations;  // return parked pieces
    bool                        isValid;
    const char*                 errorMessage;
};
```

Listing 2.6: MovePlan definition

The gantry controller executes these fields in order:

1. All `relocations` in sequence
2. The `primaryMove` along `primaryPath`
3. All `restorations` in sequence

2.7 PhysicalBoard

`PhysicalBoard` maintains a temporary occupancy model of the board that the planner mutates during simulation without affecting the engine's actual position state.

```
class PhysicalBoard {
public:
    static const int BOARD_SIZE = 8;

    PhysicalBoard();

```

```
    void      setOccupancy(Bitboard occ);
    Bitboard getOccupancy() const;

    bool isOccupied(const BoardCoord& coord) const;
    bool isOccupied(int row, int col) const;
    bool isEmpty(const BoardCoord& coord) const;
    bool isEmpty(int row, int col) const;

    static bool isValidSquare(const BoardCoord& coord);
    static bool isValidSquare(int row, int col);

    // Temporary state modification for planning
    void setOccupied(const BoardCoord& coord);
    void clearSquare(const BoardCoord& coord);
    void movePiece(const BoardCoord& from, const BoardCoord& to);

    void print() const;

private:
    Bitboard occupied;   // Bit N set => square N is occupied
};
```

Listing 2.7: PhysicalBoard class

Occupancy is stored as a single 64-bit integer (`Bitboard`) matching the engine's bit layout (bit 0 = a8, bit 63 = h1). The state-mutation methods (`setOccupied`, `clearSquare`, `movePiece`) are used exclusively during move planning on a local copy of the board; they are never called on the authoritative engine state.

Chapter 3

Path Generation

3.1 Overview

Path generation computes the ordered sequence of squares the gantry head must traverse to carry a piece from its source to its destination. Because the gantry moves beneath the board and uses a magnet to drag pieces, it must follow physically valid routes: it cannot jump over squares the way a chess move description abstracts away.

Each piece-specific generator returns a `std::vector<Path>` that may contain multiple candidate paths. The caller then selects the best candidate using `selectBestPath`.

3.2 Dispatch

`generatePath(const PhysicalMove& move)` *Function*
Parameters: `move` – the physical move including piece type, source, and destination
Returns: `std::vector<Path>`

Dispatches to the appropriate piece-specific generator based on `move.pieceType`. For `UNKNOWN` piece types, the movement pattern is inferred geometrically:

- An L-shaped pattern (`rowDiff` $\in \{1, 2\}$, `colDiff` $\in \{1, 2\}$ with $\text{rowDiff} + \text{colDiff} = 3$) $\Rightarrow$ knight path
- Equal row and column difference $\Rightarrow$ bishop (diagonal) path
- All other patterns $\Rightarrow$ rook (orthogonal) path

3.3 Piece-Specific Generators

3.3.1 Pawn

`generatePawnPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly one path

Generates a straight-line path stepping one square at a time in the direction of travel. Handles both single-step advances, double-step advances from the starting rank, and diagonal captures with the same algorithm (step in the direction of the row and column delta simultaneously).

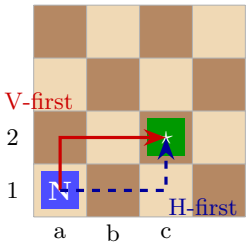
3.3.2 Knight

`generateKnightPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly two paths

Knights jump in chess, but the gantry must physically travel through intermediate squares. Two L-shaped routes are generated for every knight move:

1. **Vertical-first**: travel the longer leg along the row direction, then the shorter leg along the column direction.
2. **Horizontal-first**: travel the shorter leg along the column direction first, then the longer leg along the row direction.

Both paths are returned so the best one (fewest blockers) can be selected.



3.3.3 Bishop

`generateBishopPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly one path

Delegates to `generateDiagonalPath`, producing a single path that steps one square diagonally at a time toward the destination.

3.3.4 Rook

`generateRookPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly one path

Delegates to `generateOrthogonalPath`, stepping either horizontally or vertically depending on the direction of the move.

3.3.5 Queen

`generateQueenPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly one path

Selects diagonal movement when `|rowDiff| == |colDiff|`, orthogonal movement otherwise. Returns a single path.

3.3.6 King

`generateKingPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination coordinates
Returns: `std::vector<Path>` with exactly one path containing only the destination square

King moves are always a single square. The path contains only the destination, so the gantry moves directly without intermediate steps.

3.4 Primitive Path Helpers

Two low-level helpers are shared by multiple generators:

`generateDiagonalPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination
Returns: `Path`

Steps one square per iteration with equal row and column increments (signs determined by the direction of travel) until the destination is reached. All intermediate squares plus the destination are included.

`generateOrthogonalPath(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – source and destination
Returns: `Path`

Steps one square per iteration along either a row or a column (one of the step values is always zero). Implementation is identical to `generateDiagonalPath`; the distinction lies in what `from`/`to` pairs are passed by the callers.

3.5 Path Selection

`selectBestPath(const PhysicalBoard& board, const std::vector<Path>& paths)` *Function*
Parameters: `board` – current physical board state; `paths` – candidate paths
Returns: `Path` – the path with the fewest blocking pieces

Iterates over all candidate paths, calling `findBlockers` on each, and returns the path whose blocker count is minimal. If two paths tie, the earlier one in the vector wins. If the input vector is empty, returns an empty `Path`.

Chapter 4

Blocker Detection and Relocation

4.1 Blocker Detection

`findBlockers(const PhysicalBoard& board, const Path& path)` *Function*
Parameters: `board` – current occupancy; `path` – the path to inspect
Returns: `std::vector<BoardCoord>` – occupied intermediate squares

Scans every square in `path` *except* the destination and returns all that are occupied. The destination is excluded because captures are handled separately by the gantry controller (the captured piece is removed before the primary move begins).

4.2 Parking Spot Selection

When a blocker must be moved temporarily, it needs to be placed on an empty square that is not on the primary piece's path. The parking spot finder uses BFS expanding outward from the blocker's position to find the nearest such square.

`findParkingSpot(const PhysicalBoard& board, const BoardCoord& blockerPos, const std::set<BoardCoord>& excludeSquares)` *Function*
Parameters: `board` – current occupancy; `blockerPos` – the blocker's current position; `excludeSquares` – squares that must not be used
Returns: `BoardCoord` – nearest valid parking spot, or `BoardCoord(-1,-1)` on failure

The BFS explores the 8-connected neighbourhood of each visited square. A candidate square is accepted when it is empty and not in `excludeSquares`. The `excludeSquares` set is pre-populated with every square on the primary piece's path, ensuring that parked pieces do not block the main move.

4.3 Recursive Blocker Relocation

```
static bool planBlockerRelocation(
    PhysicalBoard&              board,
    const BoardCoord&           blockerPos,
    std::set<BoardCoord>&       excludeSquares,
    std::vector<RelocationPlan>& relocations,
    int                         depth);
```

Listing 4.1: Blocker relocation signature (internal)

This internal function handles the case where the path to the parking spot is itself blocked. The algorithm proceeds as follows:

1. Find a parking spot for the blocker via `findParkingSpot`.
2. Attempt to find a clear path to the parking spot using A* (`findClearPath`).
3. If a clear path exists, record the `RelocationPlan` and update the board state. Done.

4. If no clear path exists, identify the pieces blocking the route to the parking spot using a secondary BFS.
5. Recursively call `planBlockerRelocation` for each of those secondary blockers (depth-first).
6. After recursive relocations, retry A* to the parking spot.
7. If still blocked, try up to five alternative parking spots.
8. Return `false` if no solution is found.

A maximum recursion depth of 16 is enforced to prevent infinite loops in degenerate board configurations.

Algorithm 1 Recursive Blocker Relocation

```
procedure planBlockerRelocation(board, blockerPos, excludeSquares, relocations, depth)
    if depth > MAX_DEPTH (16) then return false
    end if
    parkingSpot ← findParkingSpot(board, blockerPos, excludeSquares)
    if parkingSpot is invalid then return false
    end if
    path ← findClearPath(board, blockerPos, parkingSpot)
    if path is not empty then
        record relocation, update board return true
    end if
    pathBlockers ← BFS to find blockers between blockerPos and parkingSpot
    for all pb ∈ pathBlockers do
        if board.isOccupied(pb) then
            if not planBlockerRelocation(board, pb, excludeSquares, relocations, depth+1) then return false
            end if
        end if
    end for
    path ← findClearPath(board, blockerPos, parkingSpot)
    if path is empty then
        try up to 5 alternative parking spots
    end if
    if path is still empty then return false
    end if
    record relocation, update board return true
end procedure
```

Chapter 5

A* Pathfinding

5.1 Overview

A* pathfinding is used exclusively for routing blocker pieces to their parking spots. Unlike primary piece paths (which follow chess movement rules), blocker paths must avoid all occupied squares and find the shortest physically traversable route.

`findClearPath(const PhysicalBoard& board, const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `board` – current occupancy; `from` – start; `to` – goal
Returns: `Path` – shortest clear path, or empty path if none exists

5.2 Algorithm Details

The implementation uses a standard A* search over the 8-connected grid:

- **Open set**: `std::priority_queue` (min-heap) of `AStarNode` values keyed on $f = g + h$.
- **Heuristic** h: Chebyshev distance (maximum of $|\Delta\text{row}|$ and $|\Delta\text{col}|$), which is admissible because the gantry can move diagonally and each step costs 1.
- **Edge cost**: Uniform cost of 1 per step regardless of direction.
- **Obstacle avoidance**: Occupied squares are skipped, except for the goal square (which may be occupied by the piece being relocated there).
- **Path reconstruction**: Follows `cameFrom` pointers from goal back to start, then reverses.

```cpp
struct AStarNode {
    BoardCoord coord;
    int        fScore;   // g + h

    bool operator>(const AStarNode& other) const {
        return fScore > other.fScore;
    }
};
```

Listing 5.1: AStarNode definition

5.3 Distance Metrics

Two distance metrics are defined for use in heuristics and path evaluation:

`chebyshevDistance(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – two squares
Returns: `int` – $\max(|\Delta r|, |\Delta c|)$

The Chebyshev (chessboard) distance is the minimum number of king moves needed. It is the A* heuristic because the gantry can step diagonally.

`manhattanDistance(const BoardCoord& from, const BoardCoord& to)` *Function*
Parameters: `from`, `to` – two squares
Returns: `int` – $|\Delta r| + |\Delta c|$

Manhattan distance counts only orthogonal steps. Available for use in contexts where diagonal travel is not permitted.

Chapter 6

Move Planning

6.1 planMove

`planMove(PhysicalBoard& board, const PhysicalMove& move)` *Function*
Parameters: `board` – current physical board state (will be read but not permanently modified); `move` – the physical chess move to plan
Returns: `MovePlan` – complete execution plan

This is the primary entry point for the board interpreter. It orchestrates all sub-systems to produce the complete `MovePlan`. The board parameter is passed by reference to allow temporary mutation during planning via a local copy; the original is not modified.

6.2 Planning Algorithm

1. **Generate candidate paths** for the primary piece by calling `generatePath(move)`.
2. **Select the best path** using `selectBestPath`, which minimises the number of blockers.
3. **Identify all blockers** on the chosen path using `findBlockers`. Intermediate squares only; the destination is excluded.
4. **Build the exclusion set** containing every square on the primary path and the primary piece's source square.
5. **Copy the board** into `tempBoard`. All subsequent mutations happen on this copy.
6. **Plan blocker relocations**: for each blocker call `planBlockerRelocation`, which recursively handles secondary blockers and appends `RelocationPlan` entries to `plan.relocations`.
7. **Apply the primary move** to `tempBoard` so the restored post-move board state is available for restoration path planning.
8. **Plan restorations**: iterate over `relocations` in reverse order (LIFO). For each relocation, call `findClearPath` from the parking spot back to the piece's original square. If no clear path exists, the path is left empty and the gantry controller falls back to a direct move.
9. **Mark the plan valid** and return.

6.3 Error Handling

`MovePlan.isValid` is set to `false` and `MovePlan.errorMessage` is populated in the following failure cases:

Warning: If `planMove` returns an invalid plan, the gantry controller must not attempt to execute it. The game controller layer is responsible for checking `MovePlan.isValid` before dispatching commands.

Condition	**Error Message**
No paths generated for piece type	`"No paths generated for move"`
`selectBestPath` returns empty	`"No valid path found"`
Blocker cannot be relocated	`"Cannot relocate blocker (position too constrained)"`

Table 6.1: MovePlan failure modes

Chapter 7

Utility Functions and Bitboard Helpers

7.1 Neighbour Enumeration

`getNeighbors(const BoardCoord& pos)` *Function*
Parameters: `pos` – a board coordinate
Returns: `std::vector<BoardCoord>` – up to 8 valid neighbouring squares

Returns all 8-connected neighbours of `pos` that lie within the board boundary. Used by both the BFS parking-spot finder and the A* pathfinder. Corner squares yield 3 neighbours, edge squares yield 5, and interior squares yield 8.

7.2 Inline Helper

`sign(int val)` *Inline Function*
Parameters: `val` – any integer
Returns: `int` – -1, 0, or 1

Returns the arithmetic sign of `val`. Used extensively in path generation to compute step directions along rows and columns.

7.3 Bitboard Utilities

These are standalone utility functions operating on `Bitboard` (`uint64_t`) values. They mirror the conventions of the Kirin engine's own bitboard layer so that occupancy information received from the engine can be consumed directly.

Function	Signature	Description
`bb_setBit`	`(Bitboard&, int)`	Set bit at square index
`bb_clearBit`	`(Bitboard&, int)`	Clear bit at square index
`bb_getBit`	`(Bitboard, int) -> bool`	Test bit at square index
`bb_countBits`	`(Bitboard) -> int`	Popcount via Kernighan's method
`bb_getLSB`	`(Bitboard) -> int`	Index of least significant set bit; -1 if zero

Table 7.1: Bitboard utility functions

`bb_setBit`, `bb_clearBit`, and `bb_getBit` are declared `inline` in the header for maximum performance. `bb_countBits` uses the bit-manipulation trick `bb &= bb - 1` to clear the lowest set bit on each iteration.

7.4 Position Parsing

`parseBoardPosition(const char* pieces)` *Function*
Parameters: `pieces` – space-separated list of piece-square tokens (e.g. `"Pe2 Nd5 Rb1"`)
Returns: `Bitboard` – occupancy bitboard

Converts a human-readable piece list into an occupancy bitboard for testing or manual board setup. Each token consists of an optional piece-type letter followed by a two-character square name. The function is tolerant of extra whitespace and skips malformed tokens.

Chapter 8

Debug Functions

8.1 printPath

`printPath(const Path& path) *Function*

Parameters: `path` – the path to display

Returns: void

Prints the sequence of squares in a path to `stdout` using FEN algebraic notation, e.g.:

```
Path (4 squares): e4 e5 e6 e7
```

8.2 printMovePlan

`printMovePlan(const MovePlan& plan) *Function*

Parameters: `plan` – the move plan to display

Returns: void

Prints a complete human-readable summary of a `MovePlan`. For invalid plans it displays the error message. For valid plans it prints the primary move, primary path, each relocation with its path, and each restoration with its path (or a note if no path was found). Example output:

```
=== Move Plan ===
Primary Move: e2 -> e4
Primary Path: Path (2 squares): e3 e4

Relocations (1):
  1. d3 -> a3: Path (3 squares): c3 b3 a3

Restorations (1):
  1. a3 -> d3: Path (3 squares): b3 c3 d3
=================
```

Chapter 9

Testing

9.1 Test Coverage

The board interpreter test suite is located in `board_interpreter_test.cpp`. It covers:

- **Coordinate conversion**: round-trip tests for `fromFEN`/`toFEN` and `toSquareIndex`/`fromSquareIndex`.
- **Path generation**: correctness tests for all six piece types including edge cases (pawn double move, knight at board corners, king move).
- **Blocker detection**: tests with zero, one, and multiple blockers at various positions on a path.
- **Parking spot finding**: empty board, fully occupied board, and paths where all nearby squares are excluded.
- **A* pathfinding**: clear board, simple obstacles, and constrained configurations.
- **Move planning**: end-to-end tests for moves with no blockers, single blockers, multiple blockers, and captures.

Additionally, `captured_piece_test.cpp` validates the interaction between the board interpreter and the gantry controller for capture moves, ensuring the captured piece is handled before the primary move path is traversed.

9.2 Running Tests

Build and run the test suite with CMake:

```
cd build
cmake .. -DBUILD_TESTS=ON
make
ctest --output-on-failure
```

Chapter 10

Integration Guide

10.1 Typical Usage Pattern

The following code illustrates the typical call sequence for executing an engine move:

```
#include "board_interpreter.h"

// 1. Receive occupancy from the engine
Bitboard occupancy = engine.getOccupancyBitboard();

// 2. Set up the physical board
PhysicalBoard board;
board.setOccupancy(occupancy);

// 3. Build the physical move descriptor
PhysicalMove move;
move.from      = BoardCoord::fromFEN("e2");
move.to        = BoardCoord::fromFEN("e4");
move.pieceType = PAWN;
move.isCapture = false;

// 4. Plan the move
MovePlan plan = planMove(board, move);

// 5. Check validity and dispatch
if (!plan.isValid) {
    fprintf(stderr, "Board interpreter error: %s\n", plan.errorMessage);
    return;
}

// 6. Hand off to gantry controller
gantryController.executePlan(plan);
```

Listing 10.1: Typical board interpreter usage

10.2 Capture Moves

For capture moves, set `PhysicalMove::isCapture = true`. The board interpreter itself does not remove the captured piece from the occupancy bitboard — it treats the destination square as empty for path-planning purposes (by excluding it from blocker detection). The gantry controller is responsible for lifting the captured piece off the board before the primary move begins, using the off-board staging area defined in the gantry configuration.

10.3 Special Chess Moves

10.3.1 Castling

Castling must be decomposed into two `PhysicalMove` operations by the game controller before being passed to the board interpreter:

1. King move: e.g. `e1 -> g1`
2. Rook move: e.g. `h1 -> f1`

Each is planned independently and the resulting `MovePlan` values are executed in sequence. The game controller layer in `game_controller.cpp` is responsible for this decomposition.

10.3.2 En Passant

En passant is handled similarly: the pawn's physical move is planned normally, and the capture of the opponent's pawn (which sits on a different square from the destination) is an additional operation the game controller schedules separately.

10.3.3 Promotion

Promotion is transparent to the board interpreter. The pawn moves to the back rank and the piece identity change is an engine-level concern; the physical gantry simply moves the pawn and the game controller then coordinates any physical piece swap with the captured-piece area.

Appendix A

Glossary

A* An informed graph search algorithm that finds the shortest path using a heuristic to guide exploration.

BFS Breadth-First Search; explores all nodes at a given distance before proceeding further. Used here for parking-spot selection.

Bitboard A 64-bit integer where each bit represents one square of the chess board. Bit set = square occupied.

Blocker A piece that sits on an intermediate square of another piece's intended movement path.

BoardCoord The `(row, col)` struct representing a physical square on the board.

Chebyshev Distance The maximum of $|\Delta r|$ and $|\Delta c|$ between two squares; equals the minimum number of king moves.

FEN Forsyth-Edwards Notation; used here to identify squares via two-character algebraic names (e.g. `e4`).

Gantry The motorised XY carriage beneath the board that moves a magnet to drag pieces.

Manhattan Distance The sum $|\Delta r| + |\Delta c|$; minimum steps if only orthogonal movement is allowed.

MovePlan The complete, ordered execution plan for one chess move on the physical board.

Parking Spot A temporary empty square to which a blocking piece is relocated while the primary move is executed.

Path An ordered sequence of `BoardCoord` values (not including source) that the gantry head traverses.

PhysicalBoard The occupancy-tracking class used during move planning; mutated on a temporary copy.

PhysicalMove A simple descriptor of a piece movement: source, destination, piece type, and capture flag.

RelocationPlan A plan to temporarily move a single blocker to a parking spot, including the route.

Restoration The return of a temporarily parked piece to its original square after the primary move completes.

Appendix B

References

- Kirin Chess Engine source: `engine.h`, `position.h`, `bitboard.h`
- Gantry Controller interface: `gantry_controller.h`
- Game Controller integration: `game_controller.h`
- A* Search Algorithm: https://en.wikipedia.org/wiki/A*_search_algorithm
- Chebyshev distance in chess contexts: https://www.chessprogramming.org/Distance
- Kernighan's bit-counting method: Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd ed., exercise 2-9.